Prompt to paste into your AI coding tool

READ THIS FIRST — do not skip

This is an in-place rebuild of the CURRENTLY OPEN project. One project, no second folder. Attach `nagdrishti-step1-scaffold.zip` alongside this prompt. Do the following, in this exact order, before touching Step 2 below:

1. **Save the logo first, if one exists.** Look for any logo/icon image files in the current project (check `src/assets`, `public`, or similar). Copy them into a new top-level folder called `brand-assets/` so they survive the deletion in the next step. If no logo file exists, skip this.

2. **Delete everything else in the current project** except `.git` (to preserve version history) and the new `brand-assets/` folder you just created. This means removing: the old `src/` folder (React/Vite code), the old `backend/` folder (the FastAPI code that was never actually connected to anything), `docker-compose.yml`, `vite.config.*`, the old `package.json`, and any other old build config. This project is being rebuilt, not patched.

3. **Extract the contents of `nagdrishti-step1-scaffold.zip` directly into the project root.** This gives you a new `backend/` folder (Django) and `frontend/` folder (Next.js) sitting where the old code used to be. This is now the entire project — one repo, two folders, correct architecture.

4. Confirm the project structure now looks like: `project-root/backend/`, `project-root/frontend/`, `project-root/`

`brand-assets/` (if a logo existed), `project-root/.git`. Nothing else should remain from before.

Checkpoint — report back to me after this step:

- Confirm old files are gone (paste the output of a top-level `ls`/`dir`)
- Confirm `backend/` and `frontend/` are now present and match what's in the scaffold zip
- Confirm whether a logo was found and saved, or whether none existed

Only proceed to Step 2 below after this is confirmed.

Context (read before making any changes)

This is a Django + Next.js project called NagDrishti AI. Step 1 (project scaffolding) is already built and verified working: Django backend with 5 apps (zones, reports, risk, routing, alerts), GeoDjango + DRF + CORS configured, a working `/api/health/` endpoint, and a Next.js frontend that successfully fetches from it. Do not redo Step 1 or change its structure.

Two steps to do now, IN ORDER. Finish and clearly report on Step 2 before starting Step 3. Do not add anything not listed below — no extra models, no extra endpoints, no UI polish yet.

STEP 2 — Data models + seed data

1. In each app (`zones`, `reports`, `risk`, `routing`, `alerts`), create the models exactly as specified below using GeoDjango fields where noted. Use `django.contrib.gis.db.models` for any spatial field.

   zones.Zone: id, name (CharField), boundary (PolygonField),

elevation_factor (FloatField), drainage_capacity (FloatField), dispatch_status (CharField, choices: Unassigned/Dispatched/Resolved)

zones.WeatherReading: id, zone (FK to Zone), rainfall_intensity_mm (FloatField), source (CharField, choices: imd_api/simulated), recorded_at (DateTimeField)

zones.TrafficReading: id, zone (FK to Zone), congestion_level (IntegerField 0-100), recorded_at (DateTimeField)

reports.Report: id, reporter_location (PointField), photo (ImageField), description (TextField), zone (FK to Zone, nullable — assigned later by spatial lookup), pothole_detected (BooleanField), pothole_confidence (FloatField), waterlogging_detected (BooleanField), waterlogging_confidence (FloatField), verification_status (CharField, choices: Pending/Verified/Rejected), created_at (DateTimeField, auto_now_add)

risk.RiskScore: id, zone (FK to Zone), score (FloatField 0-100), category (CharField, choices: Low/Medium/High/Severe), computed_at (DateTimeField, auto_now_add)

alerts.AlertLog: id, zone (FK to Zone), risk_category_at_send (CharField), channel (CharField, choices: SMS/WhatsApp), sent_at (DateTimeField, auto_now_add), status (CharField)

Note: WeatherReading and TrafficReading can live in the `zones` app since they're both per-zone readings — don't create extra apps for them.

2. Register every model in each app's `admin.py` using `django.contrib.gis.admin.GISModelAdmin` for models with spatial fields, so they're editable in the Django admin panel.

3. Run `python manage.py makemigrations` then `python manage.py migrate`. Confirm zero errors.

4. Write a management command `zones/management/commands/seed_data.py` that:

- Creates 8-10 real Nagpur ward zones with approximate real polygon boundaries (use actual Nagpur ward names — Dharampeth, Sitabuldi, Sadar, Mahal, etc. — and roughly accurate coordinates, not arbitrary shapes)
- Creates a few days of TrafficReading rows with plausible congestion values (this is explicitly meant to be simulated per the project spec — traffic has no live public source)
- Leaves WeatherReading empty for now — that gets populated in Step 3 from the real IMD/ Open-Meteo API, not seeded fake data

5. Run the seed command and confirm data appears correctly in the Django admin panel at `/admin/`.

Checkpoint — report back to me after this step:

- Confirm migrations ran clean
- Confirm you can see 8-10 real zones with correct names in `/admin/`
- Paste any error you hit

STEP 3 — Backend core logic + API

Only start this after Step 2 is confirmed working.

1. **Risk scoring function** — standalone module `risk/scoring.py`, not tangled into views. Implement: `score = 0.35*rainfall + 0.25*drainage_deficit + 0.15*elevation_factor + 0.15*historical_incidents + 0.10*report_density`, mapped to categories Low(0-25)/ Medium(26-50)/High(51-75)/Severe(76-100). Make weights named constants at the top of the file, not magic numbers, so they're easy to tune later.

2. **Routing function** — `routing/pathfinding.py` using `osmnx` to pull the Nagpur road graph and `networkx` A* for pathfinding, with edge weights penalized by the current risk score of whichever

zone each road segment falls in.

3. **Weather ingestion** — `zones/services/weather.py` that pulls real rainfall data from Open-Meteo (no API key needed) on a schedule, writes to WeatherReading with `source="imd_api"` if it's genuinely from a live feed, or `source="simulated"` if it's a fallback — never label simulated data as real.

4. **Hugging Face module** — `reports/services/detection.py` that sends a citizen's uploaded photo to the Hugging Face Inference API for pothole detection and waterlogging-presence detection, and writes the results onto the Report row. Read the API token from an environment variable, never hardcoded. If the API call fails or times out, mark the report's detection fields as null and log the failure — do not fabricate a plausible-looking result.

5. **Twilio integration** — `alerts/services/notify.py` that sends an SMS/WhatsApp alert via Twilio whenever a zone's computed risk score crosses into High or Severe, and logs it to AlertLog. Read Twilio credentials from environment variables.

6. **API endpoints**, using DRF, exactly these and no others:

    - `POST /api/reports/` (public) — citizen submits, triggers Hugging Face detection
    - `GET /api/reports/` (admin only — use `IsAdminUser`)
    - `PATCH /api/reports/{id}/verify/` (admin only)
    - `GET /api/zones/risk/` (public)
    - `PATCH /api/zones/{id}/dispatch/` (admin only)
    - `GET /api/route/?from=lat,lng&to=lat,lng` (public)
    - `GET /api/priority-queue/` (admin only)
    - `POST /api/simulate-rainfall/` (admin only)

Checkpoint — report back to me after this step:

- Confirm each endpoint responds correctly when hit with a test request (curl or Postman)
- Confirm a real Hugging Face API call actually returns a result for a test photo (not a timeout, not a fabricated fallback)
- Paste any error you hit, especially around GDAL/GeoDjango or

the Hugging Face token

Do NOT do in this prompt (out of scope, per the project spec)

- No live NMC data feed
- No IoT/sensor code
- No native mobile app
- No NLP text classification of report descriptions
- No multi-language support yet
- No frontend/UI work — that's the next prompt, after both steps above are confirmed working